

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Защита базы данных»

Выполнили:

Матвеев Илья Александрович, студент группы N3351

(подпись)

Проверил:

Карманова Наталия Андреевна, ассистент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Ход Работы	4
1.1 Задача по мониторингу	4
1.2 Задача по шифрованию	4
1.3 Задача по разграничению доступа	7
Заключение.....	11

ВВЕДЕНИЕ

Цель работы – получение навыков создания примитивных систем мониторинга, разграничения доступа и шифрования средствами СУБД.

Для достижения поставленной цели необходимо решить следующие задачи:

- Выполнить задачу по мониторингу БД – создать таблицу с логами, продемонстрировать работу логирования;
- Выполнить задачу по шифрованию данных – создать отдельную таблицу с секретной информацией, зашифровать её и продемонстрировать, что даже админ не сможет узнать информацию оттуда без ключа;
- Выполнить задачу по разграничению доступа в БД – создать роли, продемонстрировать работу системы ограничений.

1 ХОД РАБОТЫ

В данном разделе представлено выполнение всех заданий, поставленных во введении.

1.1 Задача по мониторингу

Задача по мониторингу была полностью выполнена в лабораторной работе №2, а её описание представлено в лабораторной работе №1.

Была создана отдельная таблица с логами – Logs, в которой хранятся действия, относящиеся к ученикам, менторам, заданиям и группам. Также был создан триггер, который фиксирует:

- Регистрацию нового ученика;
- Регистрацию нового преподавателя;
- Сдачу заданий;
- Регистрацию нового ментора;
- Добавление ученика в группу.

В лабораторной работе №2 были добавлены кортежи данных в таблицы Students, Groups и так далее. Все эти действия отражены в таблице с логами. Содержание таблицы с логами представлено в листинге 1.

Листинг 1 – Результат запроса `select * from Logs`

```
1      1      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
2      2      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
3      3      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
4      4      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
5      5      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
6      6      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
7      7      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
8      8      "Student"  "Регистрация нового ученика" "2025-09-23
01:10:36.719632"
```

1.2 Задача по шифрованию

Для выполнения задачи по шифрованию первоначально необходимо создать таблицу, отдельную от остальных, в которой хранятся секретные данные. Для соответствия бизнес-

логике приложения считаю уместным создать в базе данных таблицу токенов для каждого курса, по которым можно будет однозначно восстановить всю информацию о курсе.

Представим, что по каждому токену можно однозначно получить информацию о курсе (реализация будет представлена дальше), расшифровка токенов жестко ограничена при помощи пароля, а токены связаны с курсами по id курсов, при удалении курса удаляется и токен. Также токен имеет дату создания и дату удаления. Пример создания такой таблицы представлен в листинге 2.

Листинг 2 – Создание таблицы с токенами

```
CREATE TABLE CourseTokens (  
    token_id SERIAL PRIMARY KEY,  
    courseid INT NOT NULL,  
    encrypted_token BYTEA NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    expires_at TIMESTAMP,  
  
    CONSTRAINT fk_course  
        FOREIGN KEY (courseid) REFERENCES Courses(courseid)  
        ON DELETE CASCADE  
);
```

Как видно из листинга, атрибут encrypted_token задан типом bytea, что соответствует зашифрованной информации в PostgreSQL. Для того, чтобы зашифровать токен курса, будем пользоваться паролем суперпользователя – “secretPassword”, из которого получим детерменированную хэш-функцию при помощи sha256, и с полученной хэш-функцией зашифруем каждый из токенов. Пример такой работы представлен в листинге 3, используется ЯП Python. Используется алгоритм шифрования AES-256.

Листинг 3 – Шифрование токенов

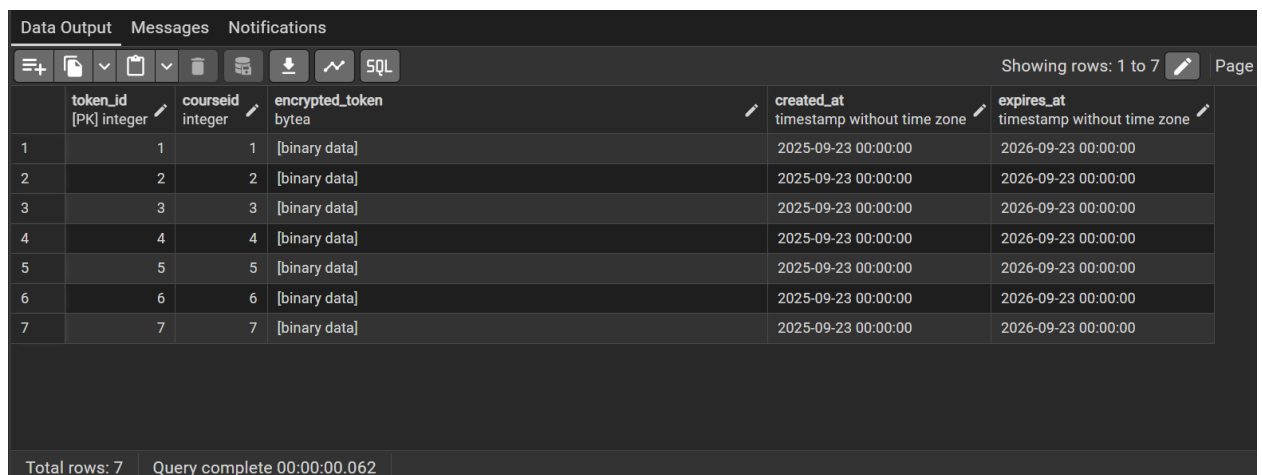
```
import hashlib  
import os  
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes  
from cryptography.hazmat.backends import default_backend  
super_password = "secretPassword"  
key = hashlib.sha256(super_password.encode()).digest()  
token = "course-1-token"  
iv = b"1234567890abcdef"  
cipher = Cipher(algorithms.AES(key), modes.CBC(iv),  
    backend=default_backend())  
encryptor = cipher.encryptor()  
padding_length = 16 - (len(token.encode()) % 16)  
padded_token = token.encode() + bytes([padding_length]) * padding_length  
encrypted_token = encryptor.update(padded_token) + encryptor.finalize()  
print("Зашифрованный токен (hex):", encrypted_token.hex())
```

Для каждого из курсов создадим токен и зашифруем его данным скриптом. Результаты внесём в таблицу с токенами. Операции вставки представлены в листинге 4.

Листинг 4 – Вставка токенов в таблицу

```
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('1', 'fa33f88c8f2b7fab64bbb7e3e9373493', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('2', '703bf3ca00402b93ae4dd458aaa4277e', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('3', 'f9da1a5f25cdfa6aa710ea7336a5e79b', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('4', 'e9169883f5603349a24925ce73c9f413', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('5', '69da2e96334d8f40f07df4bfe10e9cd1', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('6', '4fae82d9c43e18f4f7010b61f6f6787c', '2025-09-23', '2026-09-23');
INSERT INTO CourseTokens(courseid, encrypted_token, created_at, expires_at)
VALUES ('7', '62fe2b4e34fec46a13066dd5af7659f3', '2025-09-23', '2026-09-23');
```

Теперь покажем, что админы без прав суперпользователя не смогут получить доступ к зашифрованной информации. Самое простое первое действие – попробовать исполнить запрос `select * from coursetokens`, который покажет информацию, по которой невозможно понять, что зашифровано. Результат запроса представлен на рисунке 1.



	token_id [PK] integer	courseid integer	encrypted_token bytea	created_at timestamp without time zone	expires_at timestamp without time zone
1	1	1	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
2	2	2	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
3	3	3	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
4	4	4	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
5	5	5	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
6	6	6	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00
7	7	7	[binary data]	2025-09-23 00:00:00	2026-09-23 00:00:00

Total rows: 7 Query complete 00:00:00.062

Рисунок 1 – Результат запроса

То есть, вместо ожидаемого токена админ получит только бинарную информацию. Далее, админ догадывается, что он может расшифровать данные при помощи обратного алгоритма шифрования, и напишет скрипт, представленный в листинге 5.

Листинг 5 – Скрипт дешифровки информации.

```
encrypted_token = bytes.fromhex("49d97d156ef786cfcb6c01e3ffaa010e")
wrong_password = "admin123"
iv = b"1234567890abcdef"
wrong_key = hashlib.sha256(wrong_password.encode()).digest()
cipher_wrong = Cipher(algorithms.AES(wrong_key), modes.CBC(iv),
backend=default_backend())
decryptor_wrong = cipher_wrong.decryptor()

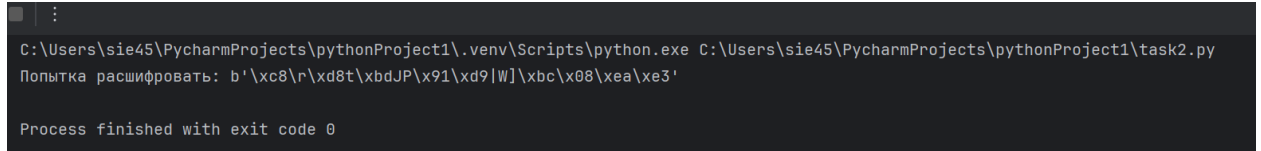
try:
    decrypted_wrong = decryptor_wrong.update(encrypted_token) +
    decryptor_wrong.finalize()
```

```

print("Попытка расшифровать:", decrypted_wrong)
except Exception as e:
    print("Ошибка при расшифровке:", e)

```

При использовании этого скрипта админ получит строку, которая явно не задумывалась как изначальный токен:



```

C:\Users\sie45\PycharmProjects\pythonProject1\.venv\Scripts\python.exe C:\Users\sie45\PycharmProjects\pythonProject1\task2.py
Попытка расшифровать: b'\xc8\r\xd8t\xbdJP\x91\xd9[W]\xbc\x08\xea\xe3'

Process finished with exit code 0

```

Рисунок 2 – Попытка расшифровки.

Таким образом, я показал, что без знания iv и пароля суперпользователя, ни один админ не сможет получить информацию о содержании зашифрованных токенов. Ключ для симметричного шифрования не хранится в информационной системе и может быть восстановлен исключительно из индивидуального пароля суперпользователя. Без знания этого пароля дешифровка невозможна.

1.3 Задача по разграничению доступа

Для разграничения доступа будем пользоваться командами grant и revoke. Так как мы уже создали представления для различных ролей в предыдущей лабораторной работе, сейчас необходимо только создать роли и передать им права с помощью вышеуказанных команд.

Код для создания ролей представлен в листинге 6.

Листинг 6 – Создание ролей

```

CREATE ROLE student LOGIN PASSWORD 'stud_pass';
CREATE ROLE teacher LOGIN PASSWORD 'teach_pass';
CREATE ROLE mentor LOGIN PASSWORD 'mentor_pass';
CREATE ROLE admin LOGIN PASSWORD 'admin_pass';

```

Далее, ограничим каждой роли полный доступ ко всем таблицам, и выдадим доступ в соответствии с представлениями. Указанные действия представлены в листинге 7.

Листинг 7 – Разграничение доступа

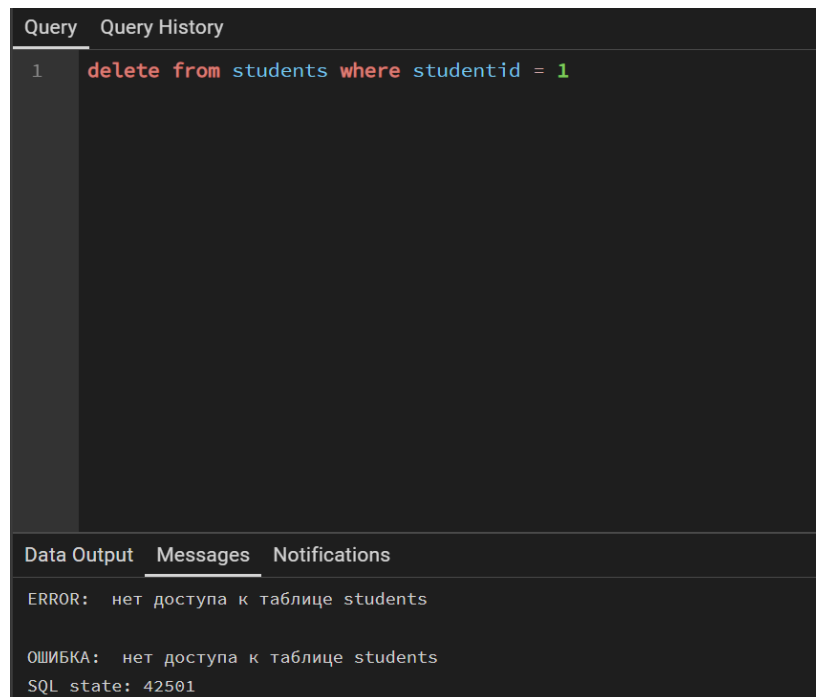
```

REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
GRANT SELECT ON Student_View TO student;
GRANT SELECT, UPDATE ON Teacher_View TO teacher;
GRANT SELECT, UPDATE ON Mentor_View TO mentor;
GRANT SELECT, DELETE ON Admin_View TO admin;

```

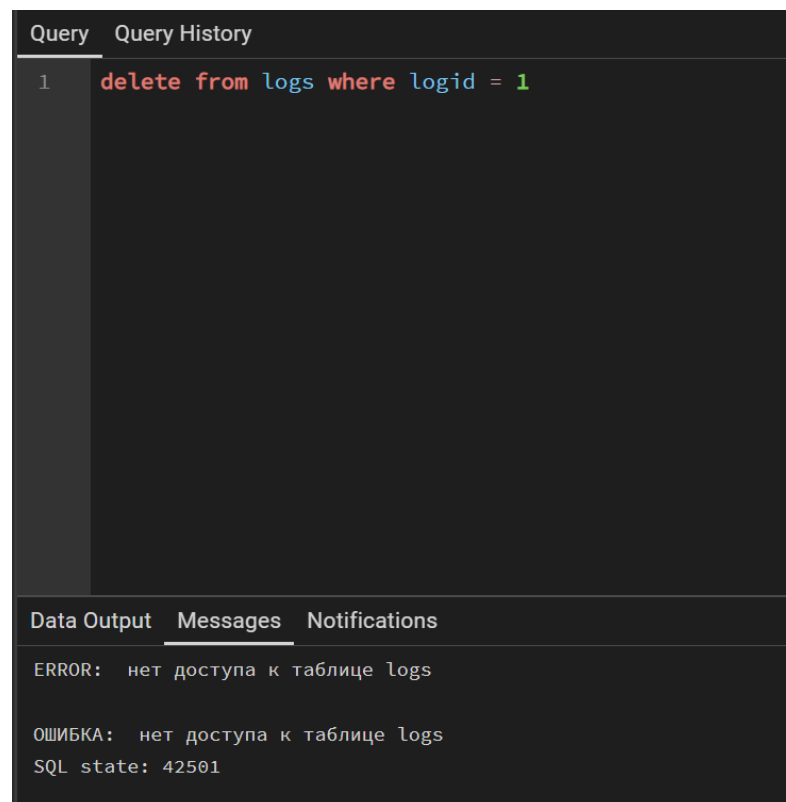
Далее, при помощи инструментов СУБД пройдем по каждому аккаунту, чтобы проверить правильность разграничения доступа.

- Выполнен вход с аккаунта student. Попытка удаление столбца из таблицы student:



The screenshot shows a database query interface with a dark theme. At the top, there are two tabs: 'Query' and 'Query History'. The 'Query' tab is active, showing a single query: `1 delete from students where studentid = 1`. Below the query, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying an error message: `ERROR: нет доступа к таблице students`. Below this, there is a Russian translation of the error: `ОШИБКА: нет доступа к таблице students` and the `SQL state: 42501`.

- Выполнен вход с аккаунта teacher. Попытка удаления столбца из таблицы logs:



The screenshot shows a database query interface with a dark theme. At the top, there are two tabs: 'Query' and 'Query History'. The 'Query' tab is active, showing a single query: `1 delete from logs where logid = 1`. Below the query, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying an error message: `ERROR: нет доступа к таблице logs`. Below this, there is a Russian translation of the error: `ОШИБКА: нет доступа к таблице logs` and the `SQL state: 42501`.

- Выполнен вход с аккаунта mentor. Попытка удаления столбца из таблицы logs:

Query
Query History

1

```
delete from logs where logid = 1
```

Data Output
Messages
Notifications

ERROR: нет доступа к таблице logs

ОШИБКА: нет доступа к таблице logs
SQL state: 42501

- Выполнен вход с аккаунта admin. Попытка просмотра столбца из таблицы logs:

Query
Query History

1

```
select * from logs
```

Data Output
Messages
Notifications

	logid [PK] integer	userid integer	userrole character varying (50)	action text	actiontime timestamp without time zone
1	1	1	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
2	2	2	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
3	3	3	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
4	4	4	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
5	5	5	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
6	6	6	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
7	7	7	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
8	8	8	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
9	9	9	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
10	10	10	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632
11	11	11	Student	Регистрация нового ученика	2025-09-23 01:10:36.719632

Total rows: 59
Query complete 00:00:00.068

Таким образом, мы можем видеть, что все роли имеют корректные права доступа, и могут просматривать только ту информацию, к которой им разрешен доступ.

ЗАКЛЮЧЕНИЕ

В рамках лабораторной работы была достигнута цель — приобретены практические навыки создания систем мониторинга, разграничения доступа и шифрования средствами СУБД PostgreSQL. Для её достижения были решены все поставленные задачи: создана таблица логов, что позволило реализовать базовый механизм мониторинга активности пользователей; разработана отдельная таблица с секретными данными (токенами курсов), информация в которой хранится в зашифрованном виде, а демонстрация показала невозможность её расшифровки без знания индивидуального пароля суперпользователя; реализована система разграничения доступа путём создания отдельных ролей и предоставления им минимально необходимых прав через представления. Таким образом, в ходе работы была подтверждена возможность защиты данных на уровне СУБД и обеспечено понимание базовых механизмов информационной безопасности в базе данных.